

Kangaroo

Showcase

$\mathcal{L}_{\text{ots-left}}^\Sigma$
$\frac{\text{OTS.ENC}(M_L, M_R):}{\begin{array}{l} K \leftarrow \Sigma.\mathcal{K} \\ C := \Sigma.\text{Enc}(K, M_L) \\ \text{return } C \end{array}}$

Cryptographic notation and game-hopping proofs
in the style of The Joy of Cryptography

1 Notation

1.1 Security parameter

The security parameter uses the lowercase lambda Greek letter.

λ	<code>\$secpa\$</code>
1^λ	<code> \$#secpa-unary\$</code>

1.2 Adversary

The adversary is identified by an uppercase calligraphic A.

$\mathcal{A}$	<code>#adv</code>
---------------	-------------------

Other adversaries may be indicated using grouped primes.

$\mathcal{A}', \mathcal{A}'', \mathcal{A}''', \dots$	<code>\$adv', adv'', adv''', ...\$</code>
--	---

1.3 Distinguishing advantage

An advantage term is characterized by a security notion, a subject attempting to achieve that notion, and the adversary attacking said notion.

$\text{Adv}_{\Sigma}^{\text{IND-CPA}}(\mathcal{A})$	<code>#advantage("IND-CPA", \$Sigma\$)</code>
$\text{Adv}_F^{\text{PRF}}(\mathcal{A}')$	<code>#advantage("PRF", \$F\$, adversary: \$adv'\$)</code>

1.4 Asymptotics

Negligible and polynomial functions of the security parameter (or a custom parameter).

$\text{negl}(\lambda), \text{negl}(n)$	<code>#negl(), #negl(param: \$n\$)</code>
$\text{poly}(\lambda), \text{poly}(n)$	<code>#poly(), #poly(param: \$n\$)</code>

1.5 Bad event

Consider a library that includes a boolean variable named `bad`, and assume that after `bad` is set to `true` it remains `true` forever.

The bad event is triggered if the library ever sets `bad` to `true`.

<code>bad := true</code>	<code>\$bad := #`true`\$</code>
--------------------------	---------------------------------

1.6 Library names

Libraries are identified by an uppercase calligraphic L and must have a name. Optionally, they may include a subject (e.g., a cryptographic construction, an algebraic structure).

$\mathcal{L}_{\text{test}}$	<code>#lib[test]</code>
$\mathcal{L}_{\text{cpa-real}}^{\Sigma}$	<code>#lib(subject: \$Sigma\$)[cpa-real]</code>
$\mathcal{L}_{\text{ddh-rand}}^{\mathbb{G}}$	<code>#lib(subject: \$GG\$)[ddh-rand]</code>

1.7 Linking and chaining

A small diamond symbol denotes linking of libraries and calling programs (i.e., adversaries).

$\diamond$	<code>#linked</code>
$\mathcal{A} \diamond \mathcal{L}_1$	<code>\$adv linked lib("1")\$</code>

Arbitrary elements can be chained together horizontally, including boxes (see [Full examples](#)).

$\mathcal{A} \diamond \mathcal{L}_1 \Rightarrow \text{true}$	<code>#chain(adv, linked, lib[1], \$=>\$, `true`)</code>
--	---

1.8 Interchangeability and indistinguishability

Libraries can be shown to be interchangeable (i.e., zero advantage for every calling program) or indistinguishable (i.e., negligible positive advantage for every polynomial-time calling program).

$\equiv, \neq$	<code>#interc.is, #interc.not</code>
$\mathcal{L}_{\text{left}} \equiv \mathcal{L}_{\text{right}}$	<code>\$lib("left") interc.is lib("right")\$</code>
$\approx, \not\approx$	<code>#indist.is, #indist.not</code>
$\mathcal{L}_{\text{left}} \approx \mathcal{L}_{\text{right}}$	<code>\$lib("left") indist.is lib("right")\$</code>

1.9 Bits and literals

Bit strings and literals are typeset in monospace using a shade of red automatically adjusted with the text color. Alternatively, the color can be set manually.

<code>10011101</code>	<code>#bit[10011101]</code>
<code>0ⁿ, 1ⁿ</code>	<code>\$bit("0")^n\$, \$bit("1")^n\$</code>
<code>9d 54 2e</code>	<code>#bit[9d] #bit[54] #bit[2e]</code>
<code>YELLOW SUBMARINE</code>	<code>#bit[YELLOW SUBMARINE]</code>
<code>example.com</code>	<code>#bit(fill: rgb("#2f6430"))[example.com]</code>
<code>127.0.0.1</code>	<code>#bit(fill: rgb("#4f5084"))[127.0.0.1]</code>

The set of bits can be used to sample binary strings.

$\{0, 1\}$	<code>#bits</code>
$\{0, 1\}^2 = \{00, 01, 10, 11\}$	<code>1 \$bits^2 = {bit("00"), bit("01"), 2 bit("10"), bit("11")}\$</code>
$K \leftarrow \{0, 1\}^\lambda$	<code>\$K <- bits^secpars\$</code>

2 Pseudocode

2.1 Algorithm names

An algorithm of a cryptographic construction is written in sans-serif style.

KeyGen	#algo[KeyGen]
$\Sigma.\text{Enc}(K, M)$	\$Sigma\.\algo("Enc")(K, M)\$

2.2 Subroutine names

The name of a subroutine is stylized in small capitals. The special END OF TIME subroutine executes just as the calling program terminates, and can be used to simplify the analysis of a bad event.

OTP.ENC	#subr[otp.enc]
END OF TIME	#end-of-time

2.3 Procedures

Algorithm and subroutine definitions in pseudocode have their names underlined, always followed by a pair of parentheses and a colon. Arguments, if present, are enclosed in parentheses.

<u>KeyGen()</u> :	#proc(algo[KeyGen])
<u>OTP.ENC(M)</u> :	#proc(subr[otp.enc], args: \$M\$)

The END OF TIME subroutine is the only exception.

<u>END OF TIME</u> :	#proc(end-of-time)
----------------------	--------------------

2.4 Comments

Code comments are typeset in italics. The default gray color can be changed.

<i>// samples uniformly at random</i>	#comment[samples uniformly at random]
<i>// fair dice roll</i>	#comment(fill: black)[fair dice roll]

2.5 Codes and libraries

A code contains lists of pseudocode lines. Both list and enum can be used; switching between the two is useful for avoiding the automatic `tight: true` setting when inserting a blank line.

Code indentation is handled automatically, following the indentation of list items.

$a \leftarrow \mathbb{Z}_n$ $A := g^a$	<pre>1 #code[2 - \$a <- ZZ_n\$ 3 - \$A := g^a\$ 4]</pre>
--	---

$$\text{Enc}(K, M):$$

$$C := K \oplus M$$

$$\text{return } C$$

```
1 #code[
2   - #proc(algo[Enc], args: $K,M$)
3     - $C := K xor M$
4     - return $C$
5 ]
```

A library is similar to a code, except that it can have an optional header containing a title, and its box has a solid border.

$$\text{SAMPLE}():$$

$$R \leftarrow \{0, 1\}^n$$

$$\text{return } R$$

```
1 #library[
2   - #proc(subr[sample])
3     - $R <- bits^n$
4     - return $R$
5 ]
```

$$\mathcal{L}_{\text{otp-real}}$$

$$\text{OTP.ENC}(M):$$

$$K \leftarrow \{0, 1\}^{|M|}$$

$$C := K \oplus M$$

$$\text{return } C$$

```
1 #library(title: lib[otp-real])[
2   - #proc(subr[otp.enc], args: $M$)
3     - $K <- bits^(|M|)$
4     - $C := K xor M$
5     - return $C$
6 ]
```

The background color of both codes and libraries can be changed. The fill parameter of library can be set to either a single color to set the background of its code area or to a pair of colors to set the header and code backgrounds, respectively.

$$a \leftarrow \mathbb{Z}_n$$

$$A := g^a$$

```
1 #set text(fill: white)
2 #code(fill: black)[
3   - $a <- ZZ_n$
4   - $A := g^a$
5 ]
```

$$\text{SAMPLE}():$$

$$R \leftarrow \{0, 1\}^n$$

$$\text{return } R$$

```
1 #set text(fill: white)
2 #library(fill: black)[
3   - #proc(subr[sample])
4     - $R <- bits^n$
5     - return $R$
6 ]
```

$$\mathcal{L}_{\text{otp-real}}$$

$$\text{OTP.ENC}(M):$$

$$K \leftarrow \{0, 1\}^{|M|}$$

$$C := K \oplus M$$

$$\text{return } C$$

```
1 #set text(fill: white)
2 #library(
3   title: lib[otp-real],
4   fill: (luma(120), black),
5 )[
6   - #proc(subr[otp.enc], args: $M$)
7     - $K <- bits^(|M|)$
8     - $C := K xor M$
9     - return $C$
10 ]
```

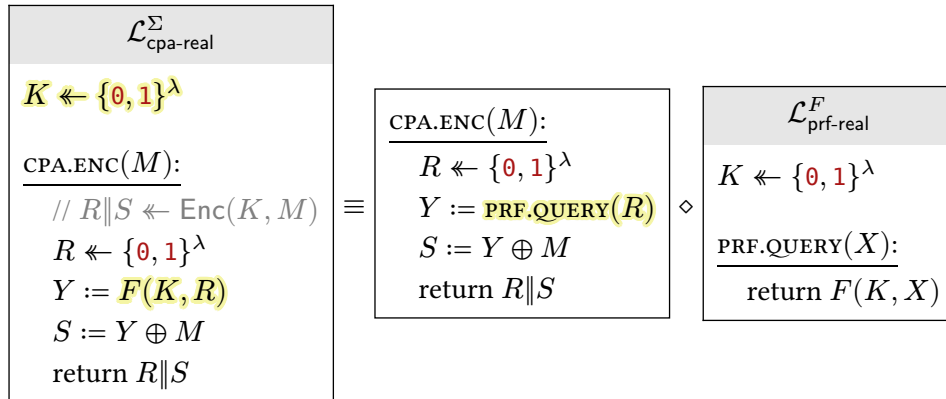
2.6 Highlighting

Highlighting can be used to indicate changes in a sequence of hybrid games or to emphasize specific terms in prose.

<code>code changes</code>	<code>#hl[code changes]</code>
$\beta \leftarrow \mathbb{Z}_n$	<code>#hl[\$beta <- ZZ_n\$]</code>
SAMPLE(): <code>// $R \leftarrow \{0,1\}^n$</code> <code>$R := 0x04$</code> return R	<pre>1 #library[2 - #proc(subr[sample]) 3 - #hl(comment[\$R <- bits^n\$]) 4 - \$R #hl[\$:= #bit("0x04")\$]\$ 5 - return \$R\$ 6]</pre>
$\mathcal{L}_{\text{otp-rand}}$ OTP.ENC(M): $R \leftarrow \{0,1\}^{ M }$ return R	<pre>1 #hl(library(title: lib[otp-rand]))[2 - #proc(subr[otp.enc], args: \$M\$) 3 - \$R <- bits^(M)\$ 4 - return \$R\$ 5])</pre>

3 Full examples

3.1 Hybrid sequence (partial)

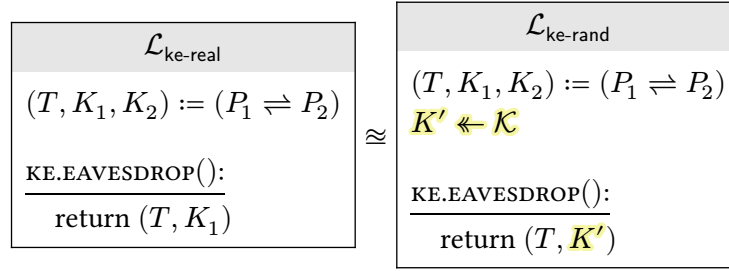


```

1 #let cpa-real = library(title: lib(subject: $Sigma$)[cpa-real])[
2   - #hl[$K <- bits^secpa$]
3
4   - #proc(subr[cpa.enc], args: $M$)
5     - #comment[$R||S <- algo("Enc")(K, M)$]
6     - $R <- bits^lambda$
7     - $Y := #hl[$F(K, R)$]$
8     - $S := Y xor M$
9     - return $R||S$
10 ]
11
12 #let prf-real = library(title: lib(subject: $F$)[prf-real])[
13   - $K <- bits^secpa$
14
15   - #proc(subr[prf.query], args: $X$)
16     - return $F(K, X)$
17 ]
18
19 #let hyb-1 = chain(
20   library[
21     - #proc(subr[cpa.enc], args: $M$)
22       - $R <- bits^lambda$
23       - $Y := #hl[$subr("prf.query")(R)$]$
24       - $S := Y xor M$
25       - return $R||S$
26   ],
27   linked,
28   prf-real,
29 )
30
31 #align(center, chain(cpa-real, intercal, hyb-1))

```

3.2 Security definition



```

1 #let ke-real = library(title: lib[ke-real])[
2   - $(T, K_1, K_2) := (P_1 harpoons.rtlb P_2)$
3
4   - #proc(subr[ke.eavesdrop])
5     - return $(T, K_1)$
6 ]
7
8 #let ke-rand = library(title: lib[ke-rand])[
9   + $(T, K_1, K_2) := (P_1 harpoons.rtlb P_2)$
10  + #hl[$K' <- cal(K)$]
11
12  - #proc(subr[ke.eavesdrop])
13    - return $(T, #hl[$K'$])$
14 ]
15
16 #align(center, chain(ke-real, indist.is, ke-rand))

```

3.3 Distinguishing advantage

$$\text{Adv}_{\Sigma}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr[\mathcal{A} \diamond \mathcal{L}_{\text{cpa-real}}^{\Sigma} \Rightarrow \text{true}] - \Pr[\mathcal{A} \diamond \mathcal{L}_{\text{cpa-rand}}^{\Sigma} \Rightarrow \text{true}] \right| \leq \text{negl}(\lambda)$$

```

1 $
2 advantage("IND-CPA", Sigma) = lr(
3   |Pr[adv linked #lib(subject: $Sigma$)[cpa-real] => #`true`]
4   - Pr[adv linked #lib(subject: $Sigma$)[cpa-rand] => #`true`]|,
5   size: #150%
6 ) <= negl()
7 $

```